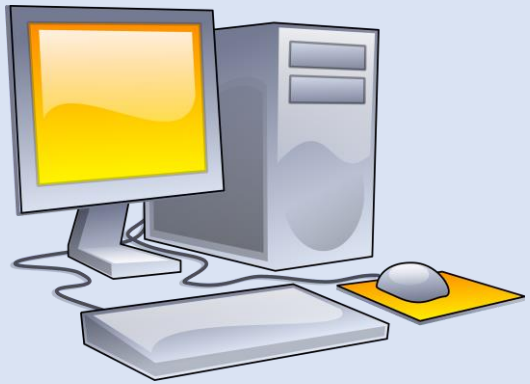
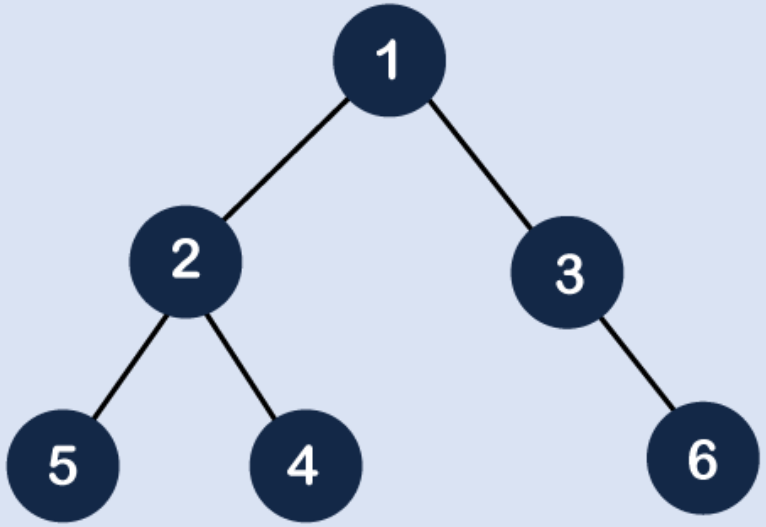
Heaps

COP 3502
Spring 2025
Department of Computer Science
University of Central Florida
Dr. Steinberg



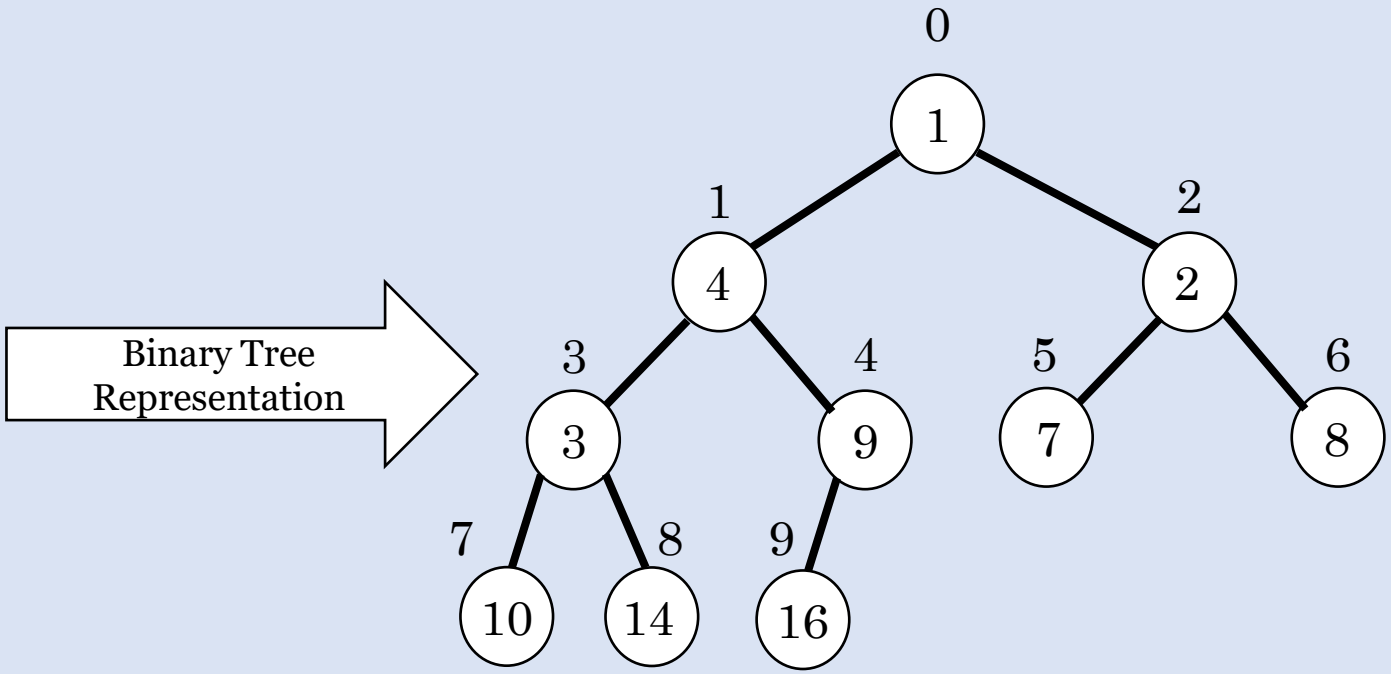


What is a heap?

- A (binary) heap is an array which can be seen as a nearly complete binary tree and where the keys must satisfy the heap property (more to come).
- Example

Array arr

1	4	2	3	9	7	8	10	14	16
0	1	2	3	4	5	6	7	8	9



Two Ways of Implementation

- Binary Tree Implementation
  - This can be a bit painful to maintain
  - However, we will draw some abstractions of the Binary Tree structure
  - CS Majors! This is a good challenge if you would like to implement Heaps using Binary Trees.
- Array Implementation
  - This is a lot simpler than the Binary Tree. We will focus on this implementation.

The Typedef Struct for the Heap

```
typedef struct{

    int *heaparray;
    int capacity;
    int size;

}heap_t;
```

The Dynamic Array

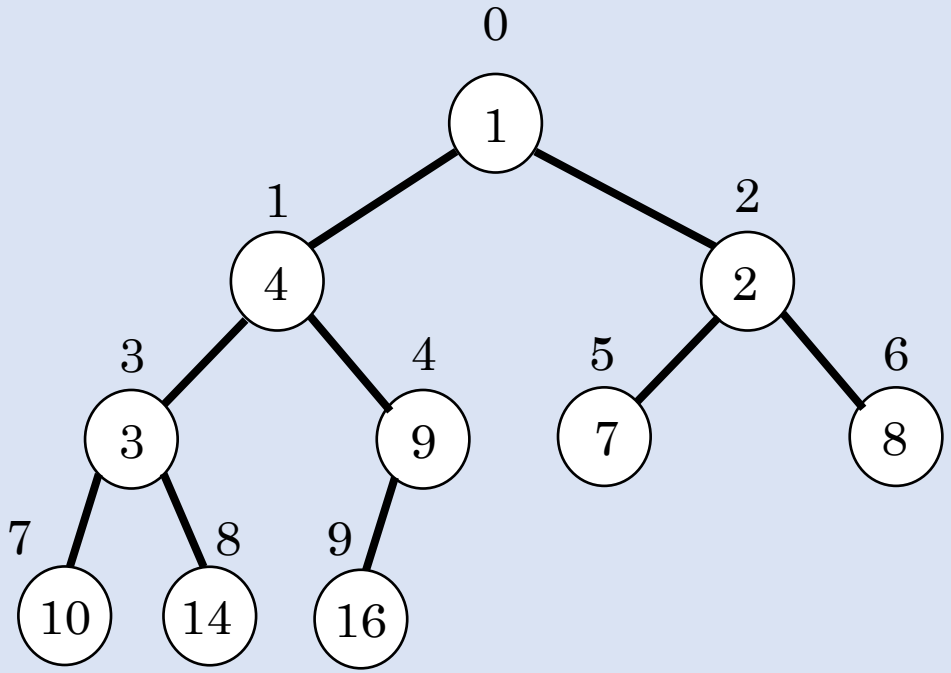
Max Size of Array

Current Size of Array

Some Array Facts to Understand in C

- Parent of index *key*:

```c
int parent(int key)
{
    return (key - 1) / 2;
}
```



Example 1: Parent of index 4 (with value 9)

$$\text{parent} = \frac{(4-1)}{2} = \frac{3}{2} = 1.5 = 1$$

Example 2: Parent of index 5 (with value 7)

$$\text{parent} = \frac{(5-1)}{2} = \frac{4}{2} = 2$$

Some Array Facts to Understand in C

- Left Child of Index *key*:

```
int leftChild(int key)
{
    return 2 * key + 1;
}
```

- Right Child of Index *key*:

```
int rightChild(int key)
{
    return 2 * key + 2;
}
```

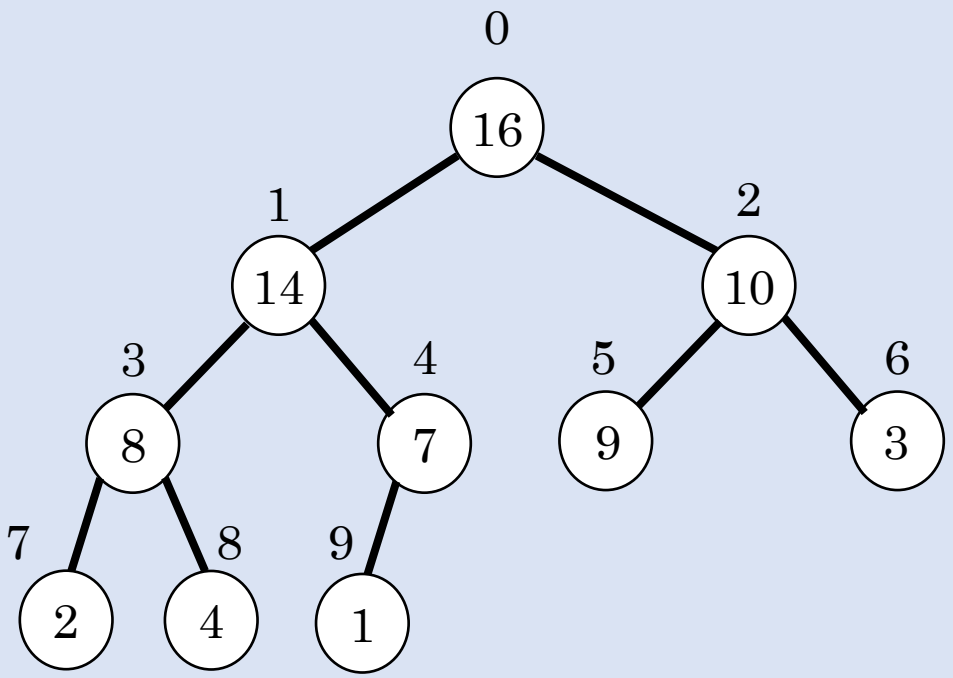
What is this “heap property”?

- There are two kinds of binary heaps:
  - max-heap
    - for any node $i \neq root$: $arr[i] \leq arr[parent(i)]$
  - min-heap
    - for any node $i \neq root$: $arr[i] \geq arr[parent(i)]$
- In both kinds, the values in the nodes satisfy a heap property, the specifics of which depend on the kind of heap (max-heap or min-heap).
- We will primarily focus on the **min-heap** in this course!

Max-Heap Example

height of a node – no. of edges on the longest simple path from the node to a leaf

height of the heap– is the height of the root node

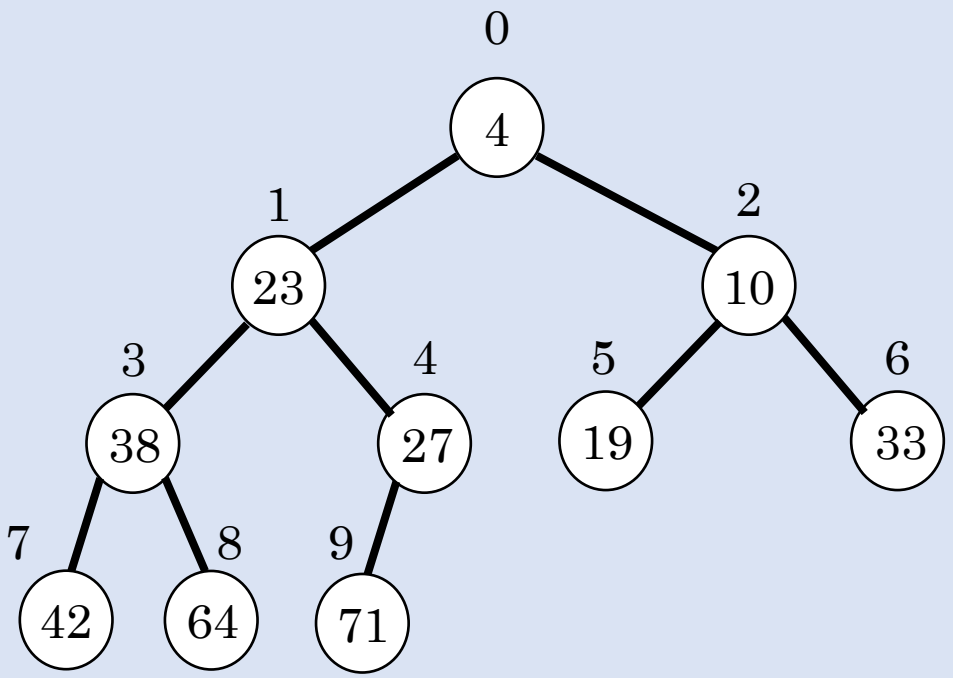


Min-Heap Example

This is the heap we will use throughout the course!!

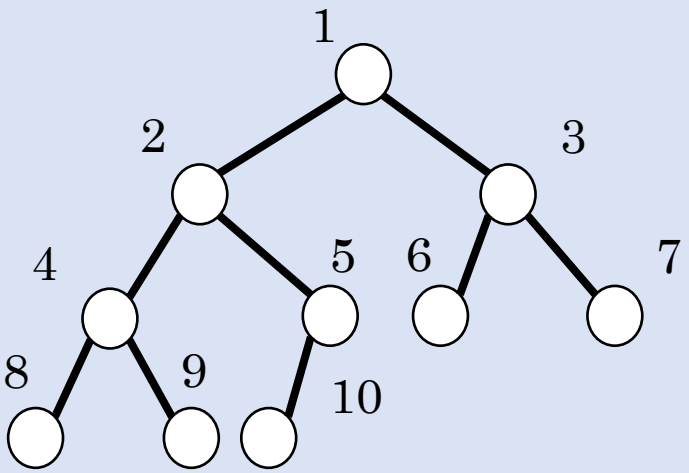
height of a node – no. of edges on the longest simple path from the node to a leaf

height of the heap– is the height of the root node



An interesting property about heaps

- In a heap of size n, the height of the heap is $h = \lfloor \log n \rfloor$
- That means $h = O(\log n)$ (AMAZING!)

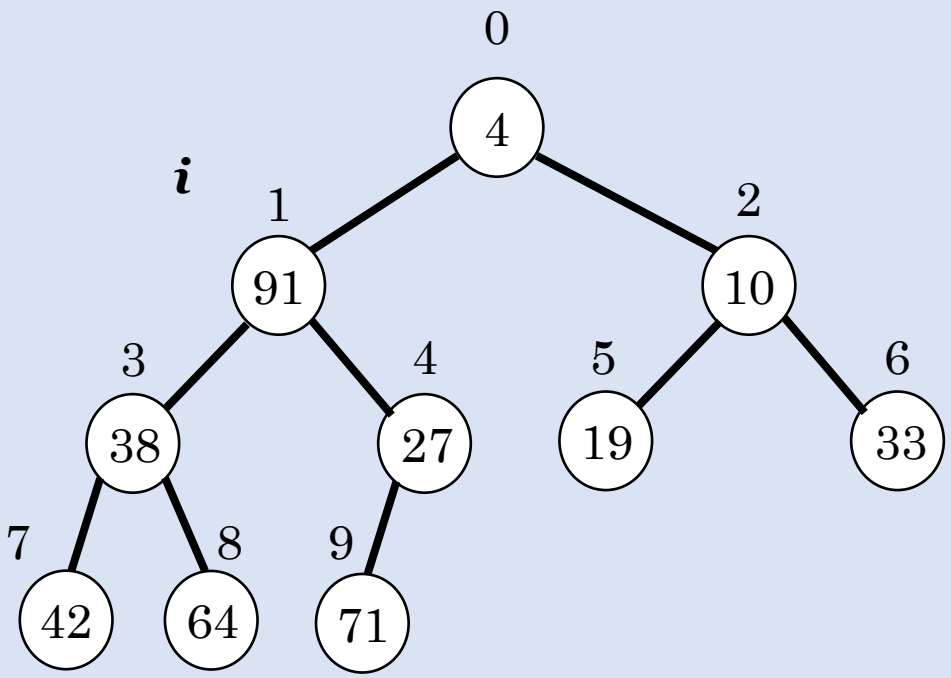


$$2^i \leq n < 2^{i+1}$$
$$height\ i = \lfloor \log n \rfloor$$
Height of the heap is O($\log n$)

Maintaining the Heap Property

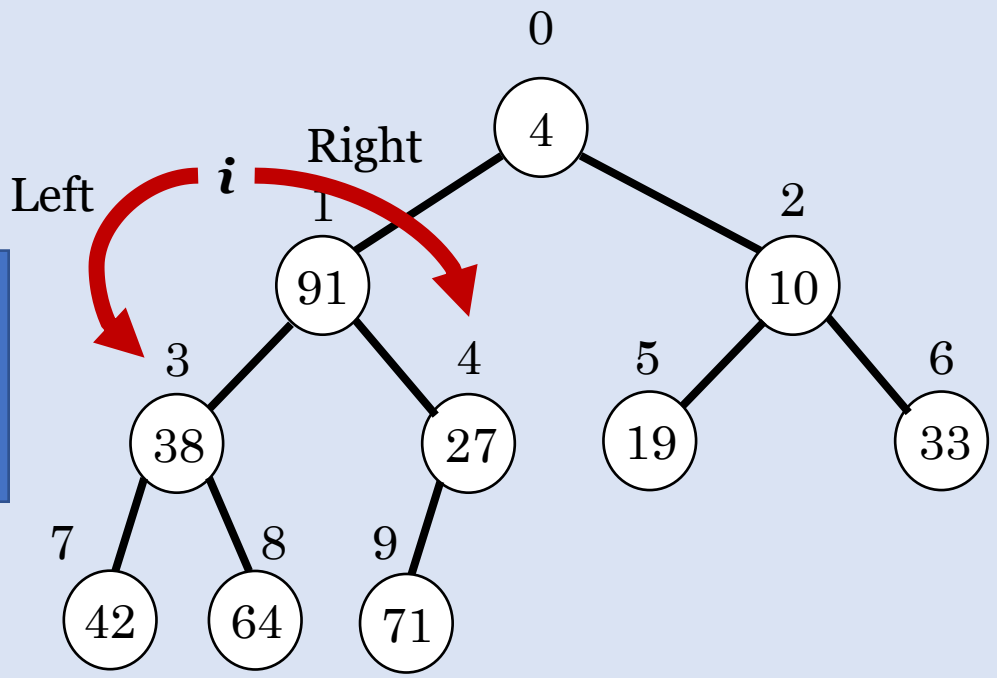
- Heapify
- Input: An array and index *i* such that
  - Subtrees rooted at the nodes left and right of *i* are max-heaps
  - The key in the node arr[*i*] might be smaller than the keys in the children
- Output: A rearrangement of the elements in the subtree rooted at node *i* such that it becomes a max-heap.

Example heapify(*i*)



Example heapify(*i*)

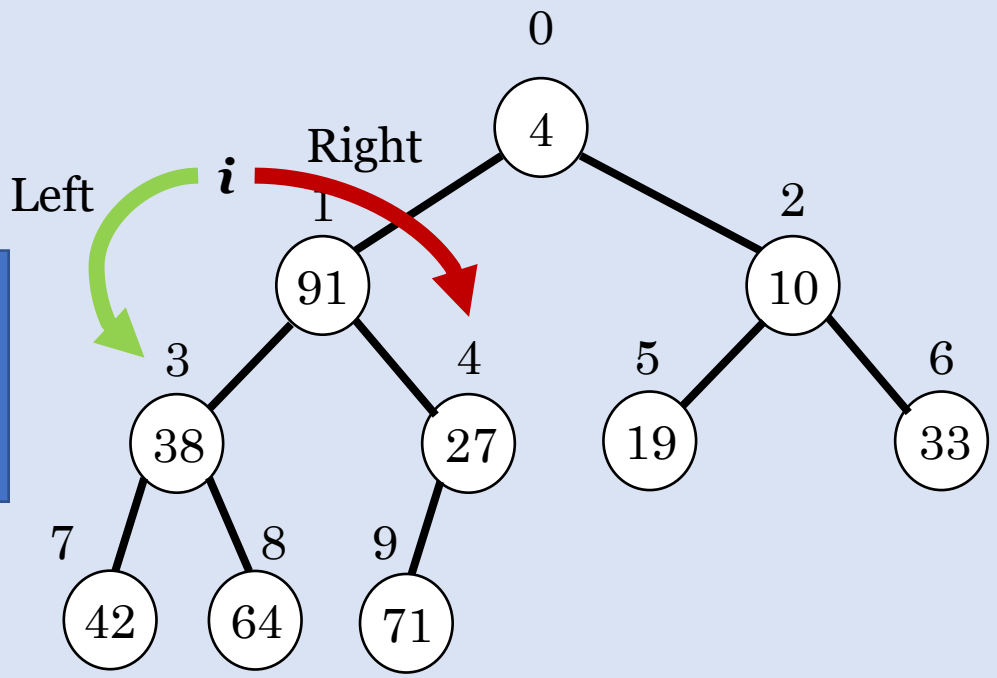
Grab the respective left child and right child indexes based on index *i*.



i = 1
Smallest = 1

Example heapify(*i*)

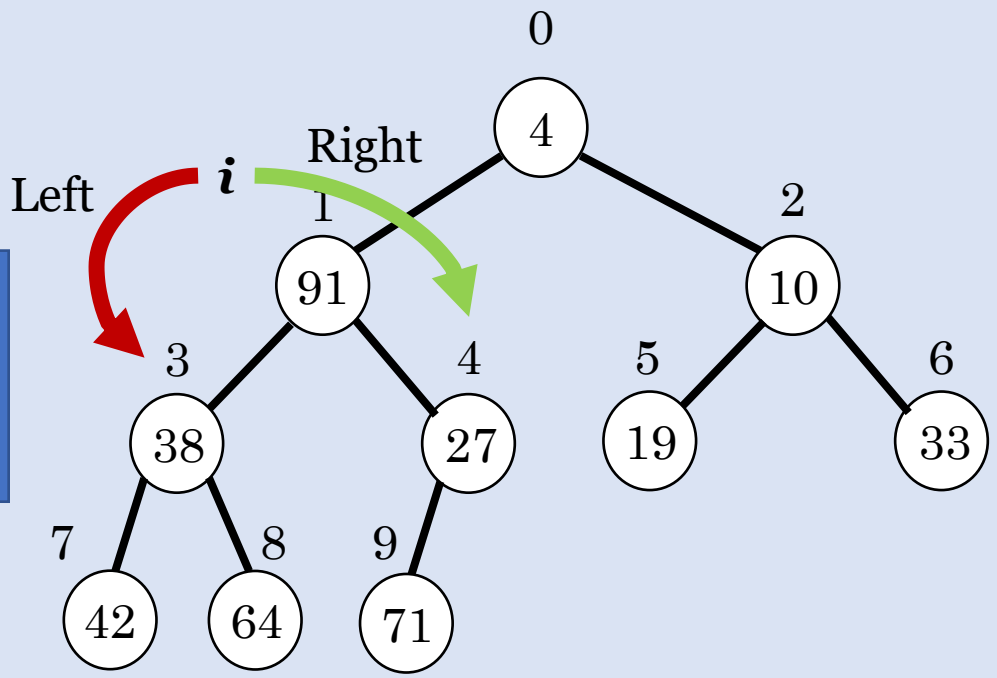
Start with left child first! Compare the values 91 and 38. The left child has a smaller value than index i!



i = 1
Smallest = 3
Left = 3
Right = 4

Example heapify(*i*)

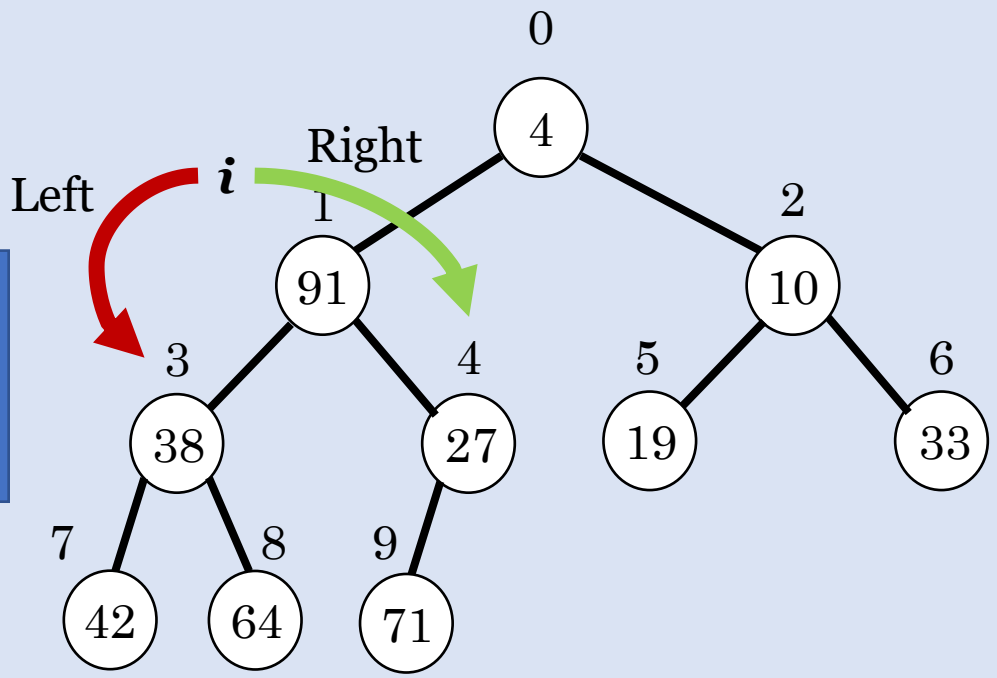
The new smallest value is 38. Now check to see if the right child is smaller!



i = 1
Smallest = 3
Left = 3
Right = 4

Example heapify(*i*)

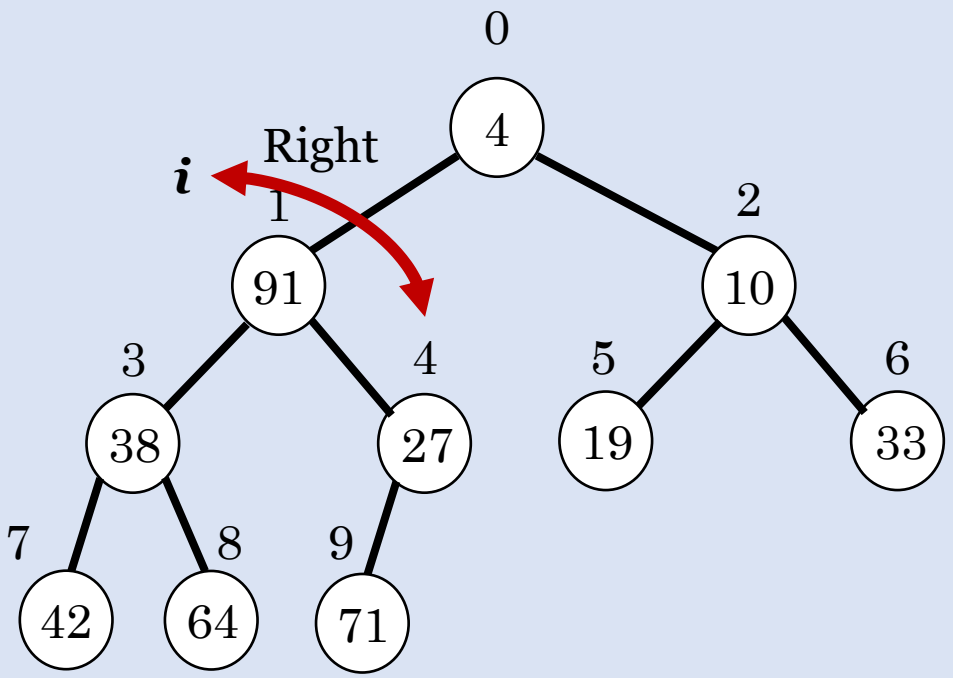
The right child is smaller!



i = 1
Smallest = 4
Left = 3
Right = 4

Example heapify(*i*)

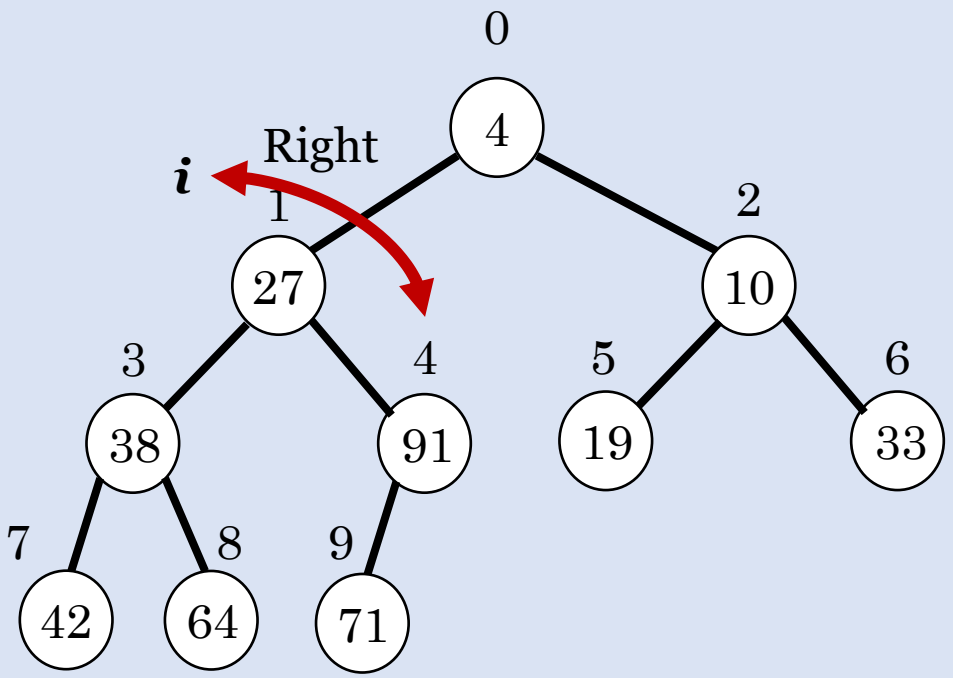
Swap the smallest value with the parent!



i = 1
Smallest = 4
Left = 3
Right = 4

Example heapify(*i*)

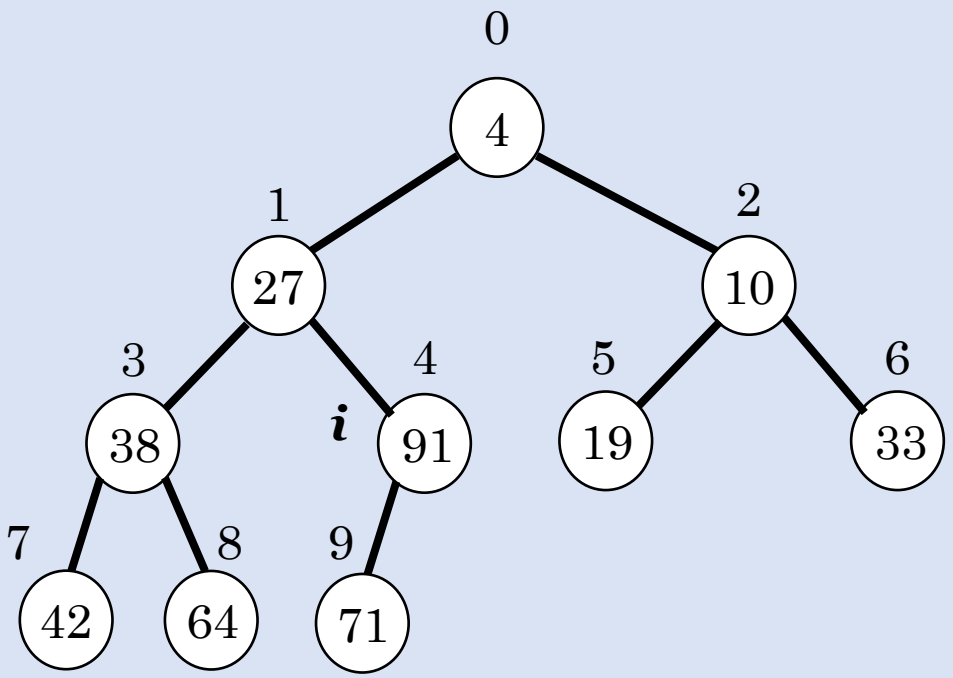
Swap the smallest value with the parent!



i = 1
Smallest = 4
Left = 3
Right = 4

Example heapify(*i*)

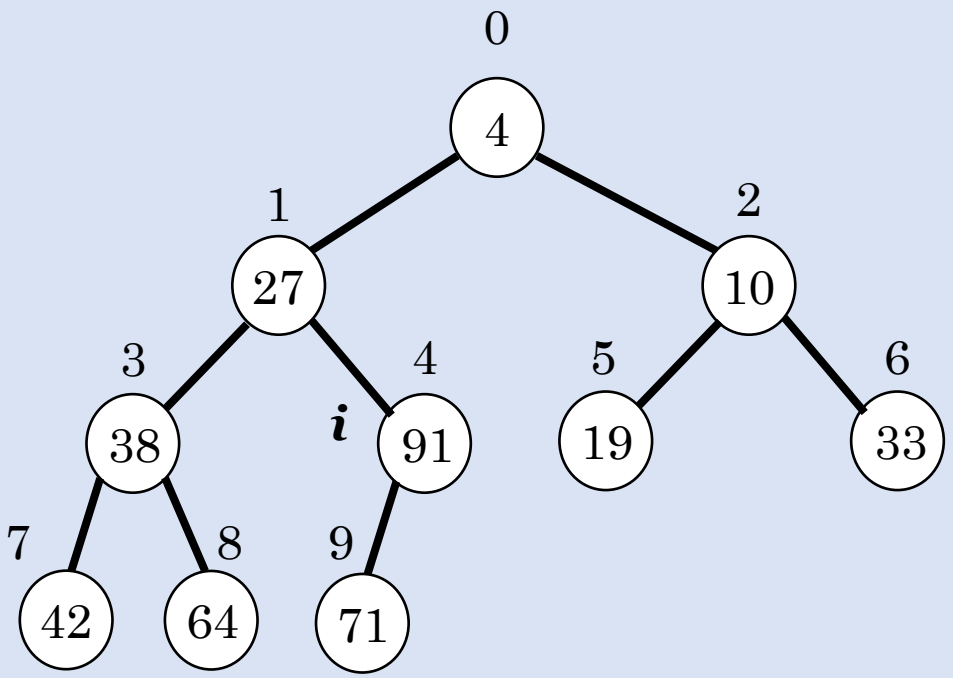
Call Heapify Again! i moves to the recent smallest index value.



i = 4

Example heapify(*i*)

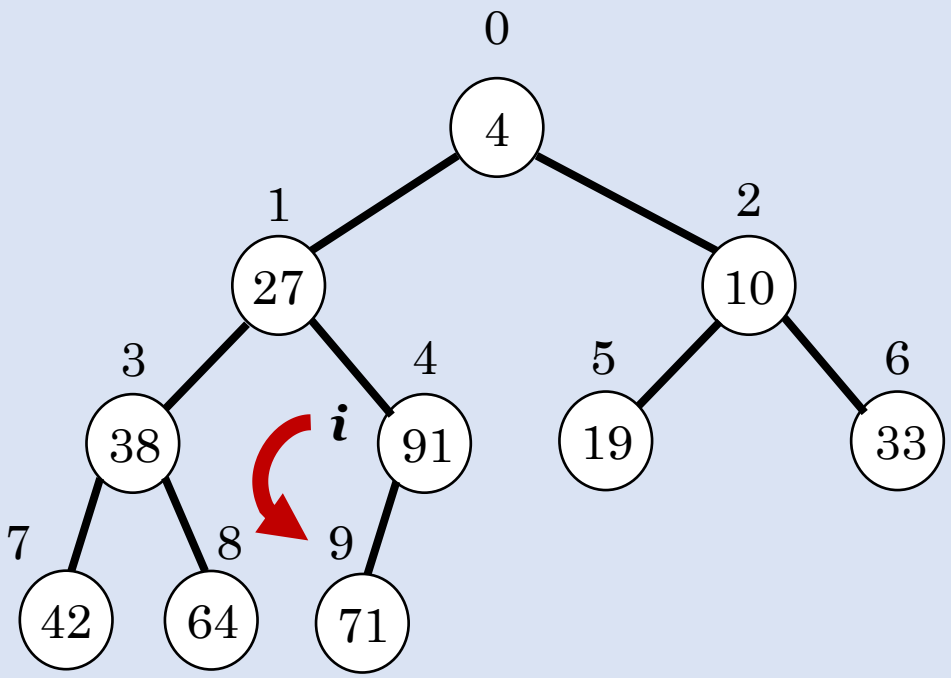
Call Heapify Again! i moves to the recent smallest index value.



i = 4
Smallest = 4

Example heapify(*i*)

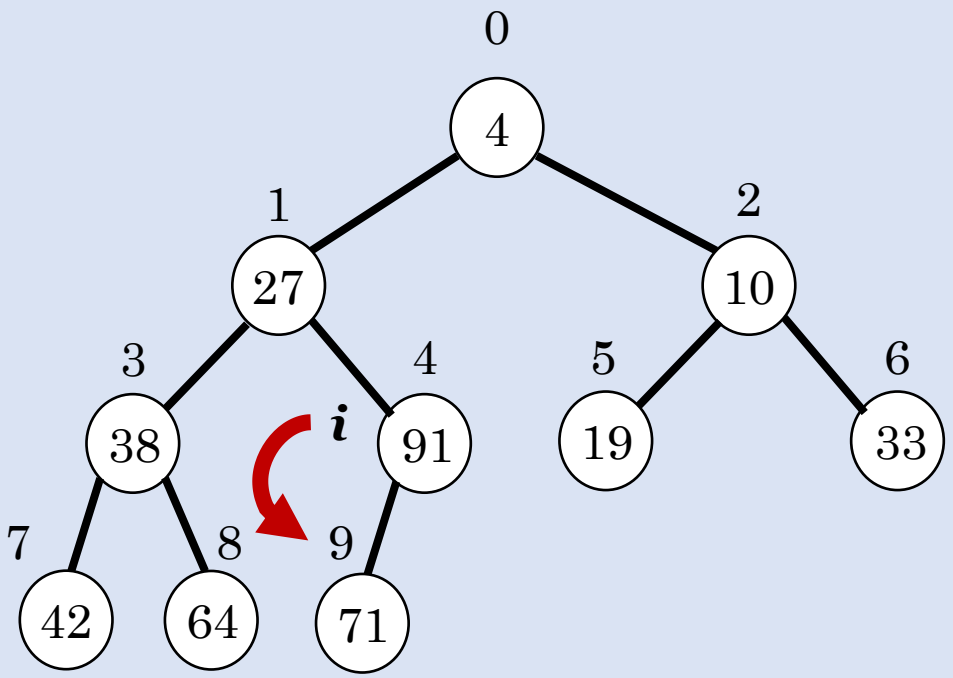
Check to see if left child value is smaller than parent!



i = 4
Smallest = 4
Left = 9
Right = 10 *There is no index 10 in this heap array!*

Example heapify(*i*)

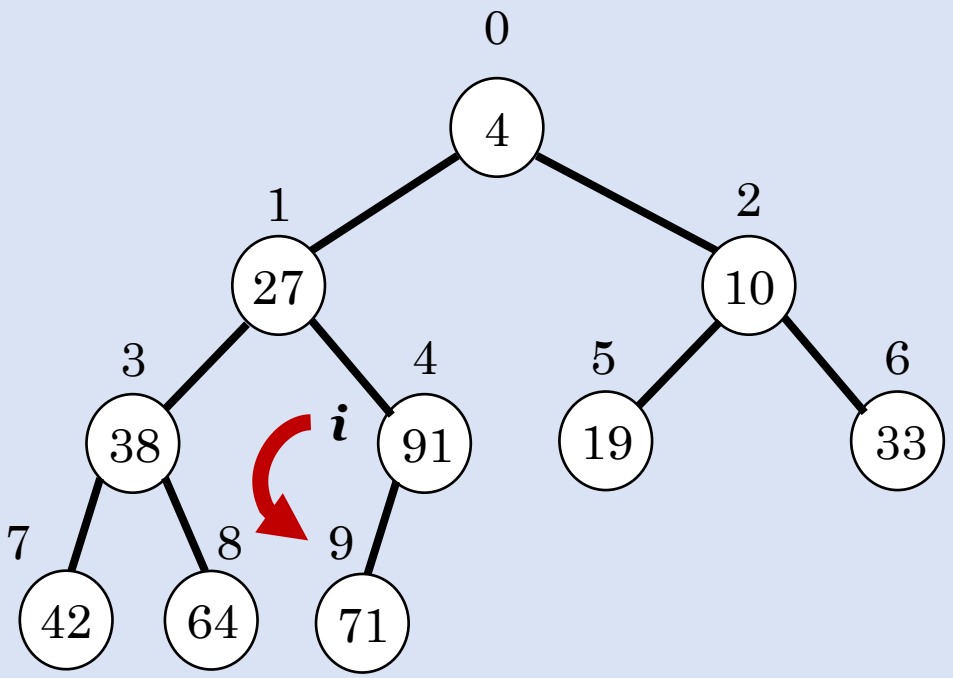
Left child value is smaller!



i = 4
Smallest = 9
Left = 9
Right = 10 *There is no index 10 in this heap array!*

Example heapify(*i*)

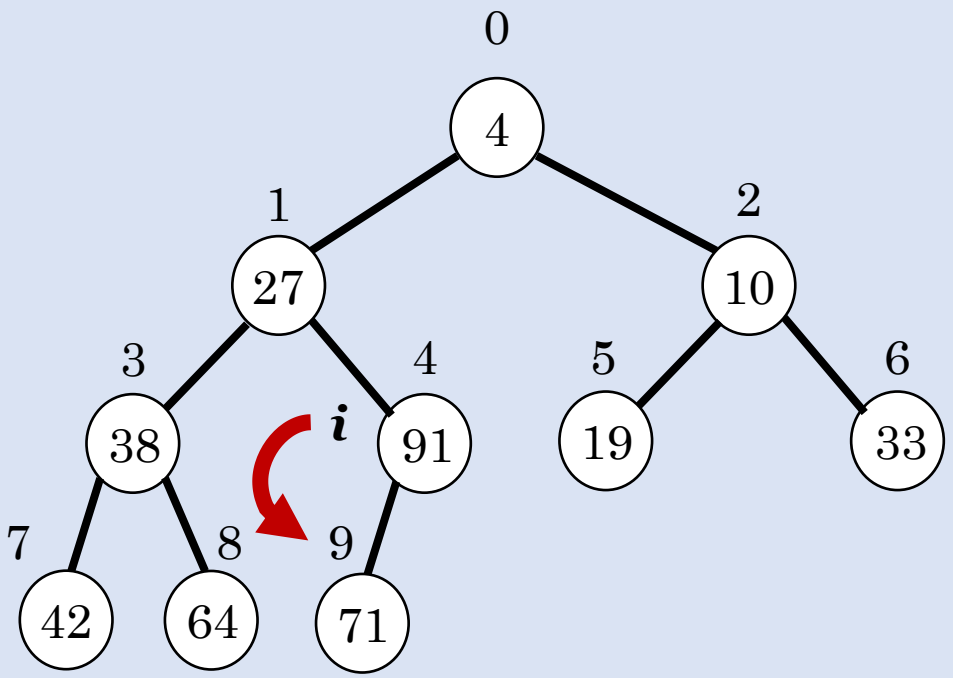
Right child value does not exist!



i = 4
Smallest = 9
Left = 9
Right = 10 *There is no index 10 in this heap array!*

Example heapify(*i*)

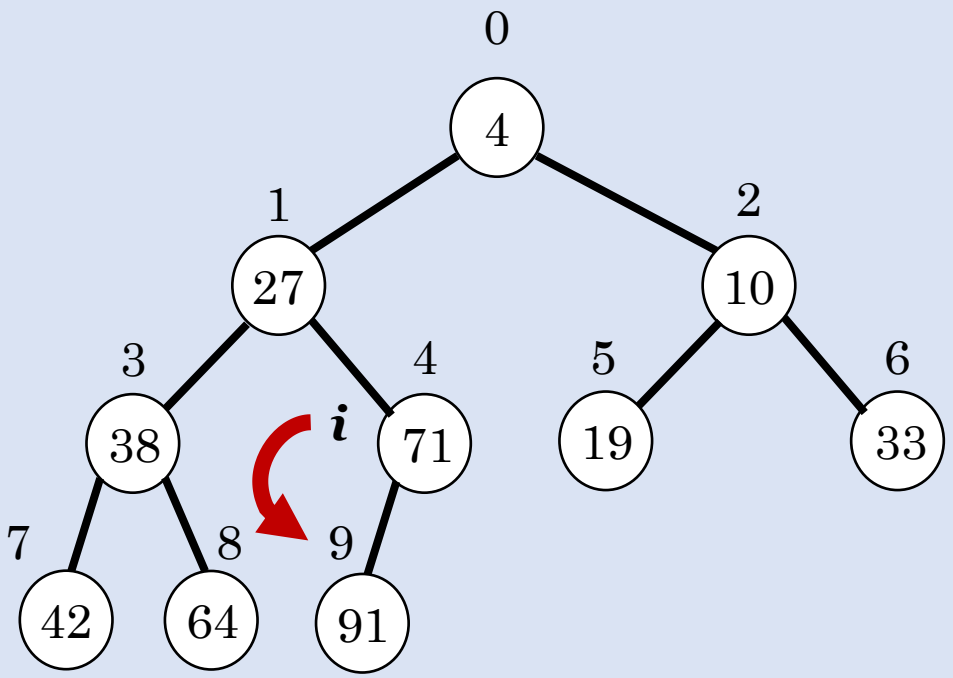
Swap the smallest value with the parent!



i = 4
Smallest = 9
Left = 9
Right = 10 *There is no index 10 in this heap array!*

Example heapify(*i*)

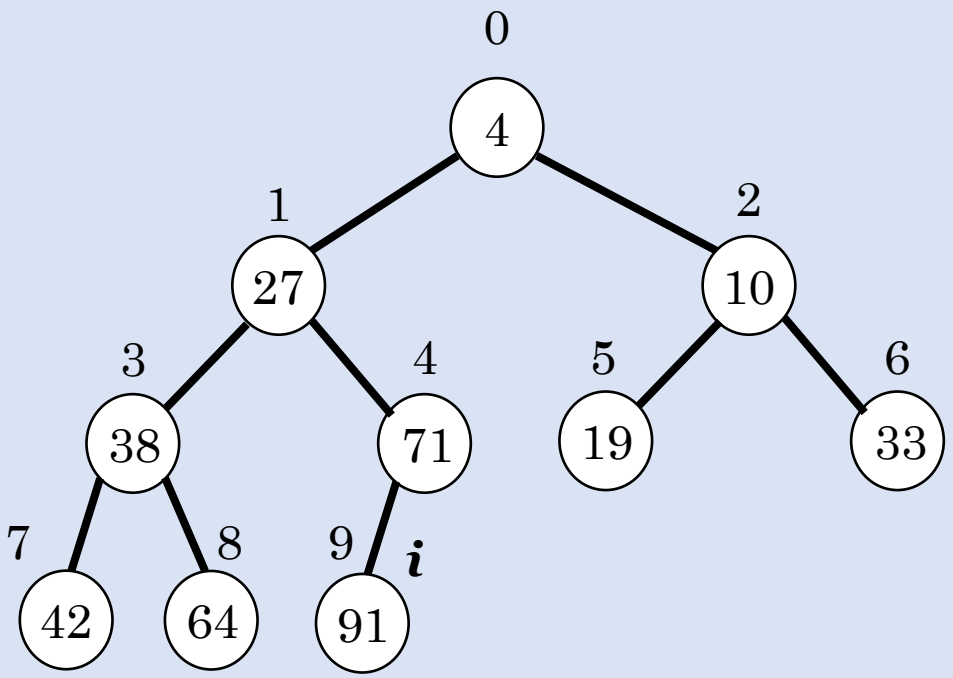
Swap the smallest value with the parent!



i = 4
Smallest = 9
Left = 9
Right = 10 *There is no index 10 in this heap array!*

Example heapify(*i*)

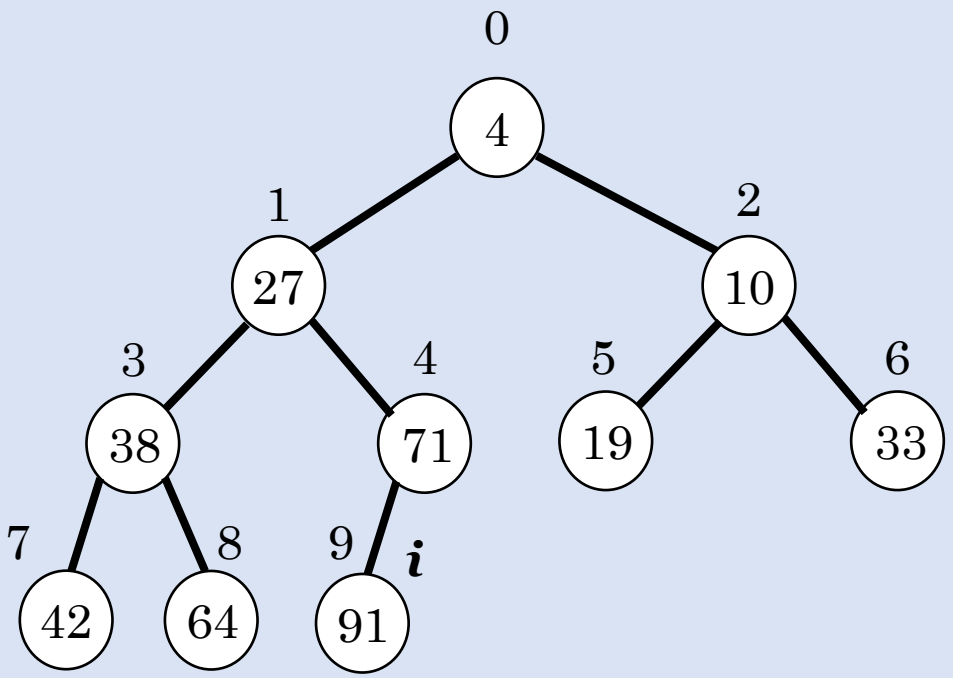
Call Heapify Again! i moves to the recent smallest index value.



i = 9
Smallest = 9

Example heapify(*i*)

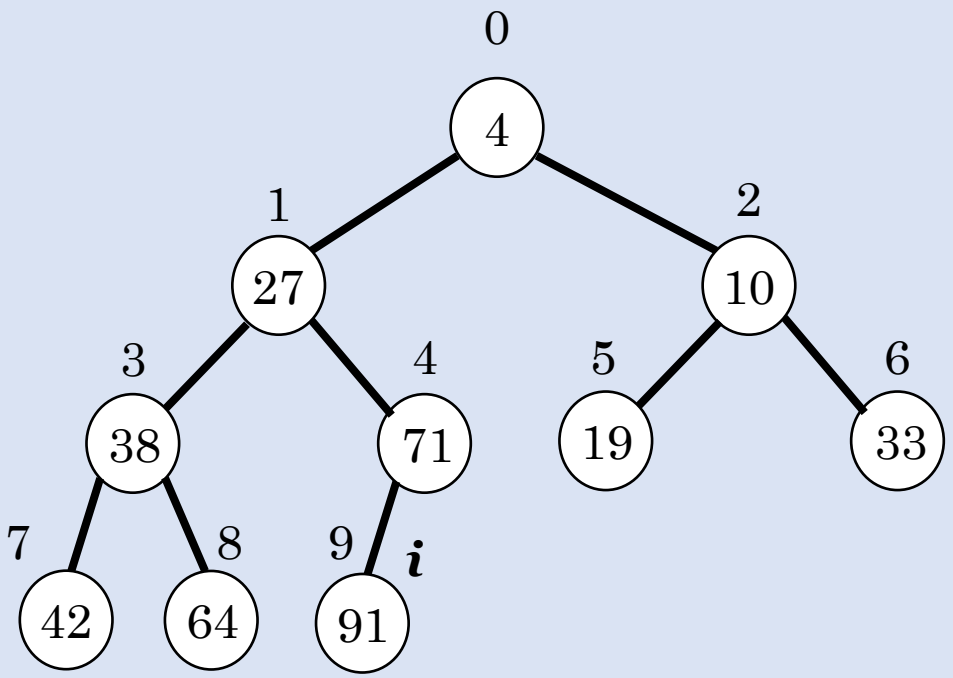
Call Heapify Again! i moves to the recent smallest index value.



i = 9
Smallest = 9

Example heapify(*i*)

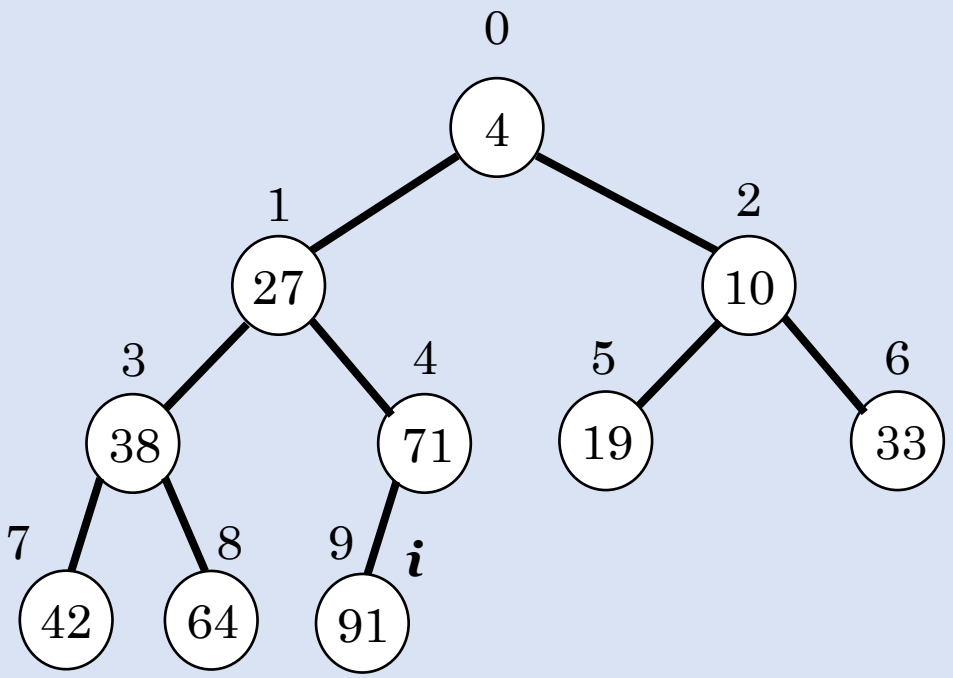
Get left and right child indexes!



i = 9
Smallest = 9
Left = 19
Right = 20
Left and Right indexes don't exist in the array!

Example heapify(*i*)

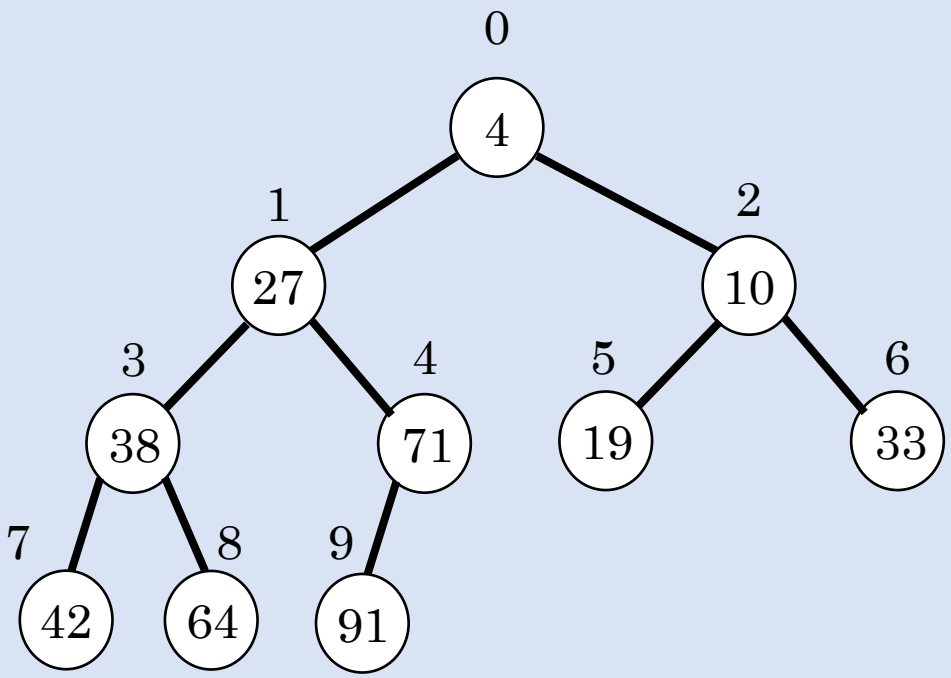
Since the indexes do not exist, the heapify operation stops! The min-heap is now satisfied!



i = 9
Smallest = 9
Left = 19
Right = 20
Left and Right indexes don't exist in the array!

Example heapify(*i*)

Final min-heap



Implementation of heapify

```
void heapify(int * arr, int n, int i)
{
    //find the smallest between the left and right child
    int left = leftChild(i);
    int right = rightChild(i);
    int smallest = i;

    //make sure index is in range first and then compare
    if(left < n && arr[left] < arr[smallest])
        smallest = left;

    //make sure index is in range first and then compare
    if(right < n && arr[right] < arr[smallest])
        smallest = right;

    //if a smaller value was found, swap with parent and heapify
    if(smallest != i)
    {
        swap(arr, i, smallest);
        heapify(arr, n, smallest);
    }
}
```

Running Time Analysis of Heapify

- The running time on some subtree of size *n* rooted at node *i* is constant time ($O(1)$) since it just performs swap mainly.
- The children's subtrees each have a size at most $2n/3$
  - The worst-case scenario happens when we start at the bottom level of the tree, and it is half full.
- Overall, this can be described by the recurrence relation
  - $T(n) \leq T\left(\frac{2n}{3}\right) + O(1)$
  - This recurrence can be solved by a special technique called Master Theorem (you will learn this technique in CS2) which results in the recurrence being expressed as $T(n) = O(\log n)$
  - We can also characterize the running of heapify of a node of some height *h* as *O(h)*.

Operations of Heap

- The Heap Performs the following operations

1. insert
2. decreaseKey
3. extractMin
4. delete
5. getMin
6. heapify (which was already discussed previously)

Insertion Operation

- Insertion into a min-heap is very simple.
- First, we insert the value at the next available spot in the heap array.
- Once the value has been inserted, we have to ensure the heap property is satisfied.
  - This requires checking the value of the parent node!
  - If the parent node is bigger than the new child node, perform a swap between the two nodes.
  - Increment the index of interest up the parent node and perform the same set of steps until the parent node is smaller than the child node.

Decrease Key Operation

- The decreaseKey operation updates the value stored at an index to a smaller value.
  - Note: It is important to check to see if the new value is smaller first! Otherwise, the min-heap will be broken!
- Once the value is updated (decreased). Fix the heap to ensure the min-heap property.

Extract Min Operation

- Removes the smallest value from the heap.
- That value is located at the root of the binary tree!
  - In other words, the first element of the heap array!!
- After grabbing the first element, update the heap array by grabbing the last element and moving up to the root of the binary tree.
- This will allow us to call heapify and properly ensure the heap maintains the min-heap property.
- There are 2 special exceptions to consider.
  - If the heap is empty, just return the largest possible value in the programming language.
  - If the heap array only has one element, just return that value and do not invoke the heapify operation.

Delete Operation

- Like insertion, a very simple operation.
- The value of the index we are removing, we have to update the value to the smallest possible value (based on the programming language).
- This will result in heap to move the updated value to the root of the heap.
- Once the value has made it to the root of the binary tree, just remove it using extractMin()
  - This will cause heapify to invoke.

Get Minimum Operation

- The getMinimum operation returns the smallest value in the heap.
- This value is located at the root of the binary tree.
  - This is the first element of the heap array.

Worst-Case Running Times for Heap Operations

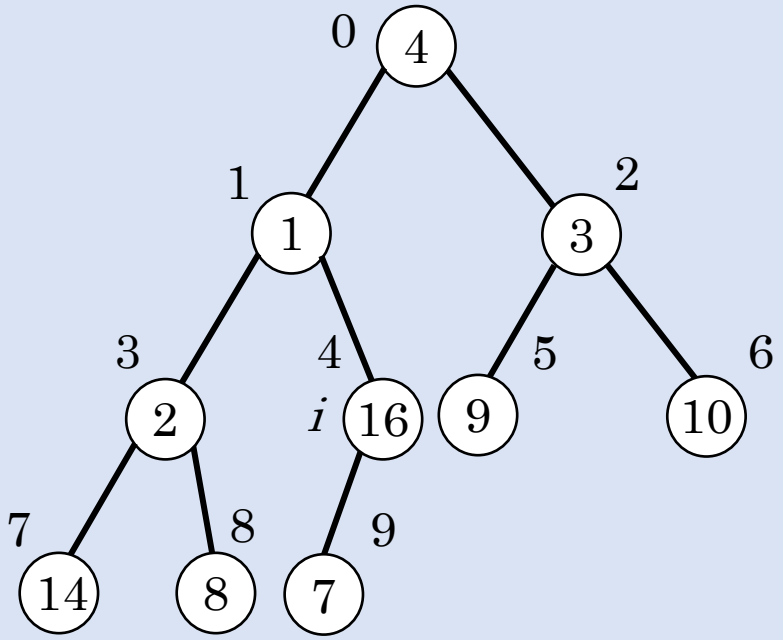
Operation	Worst-Case Running Time
Insertion	$O(h) = O(\log n)$
Decrease Key	$O(h) = O(\log n)$
Extract Minimum	$O(h) = O(\log n)$
Deletion	$O(h) = O(\log n)$
Heapify	$O(h) = O(\log n)$
Get Minimum	$O(1)$

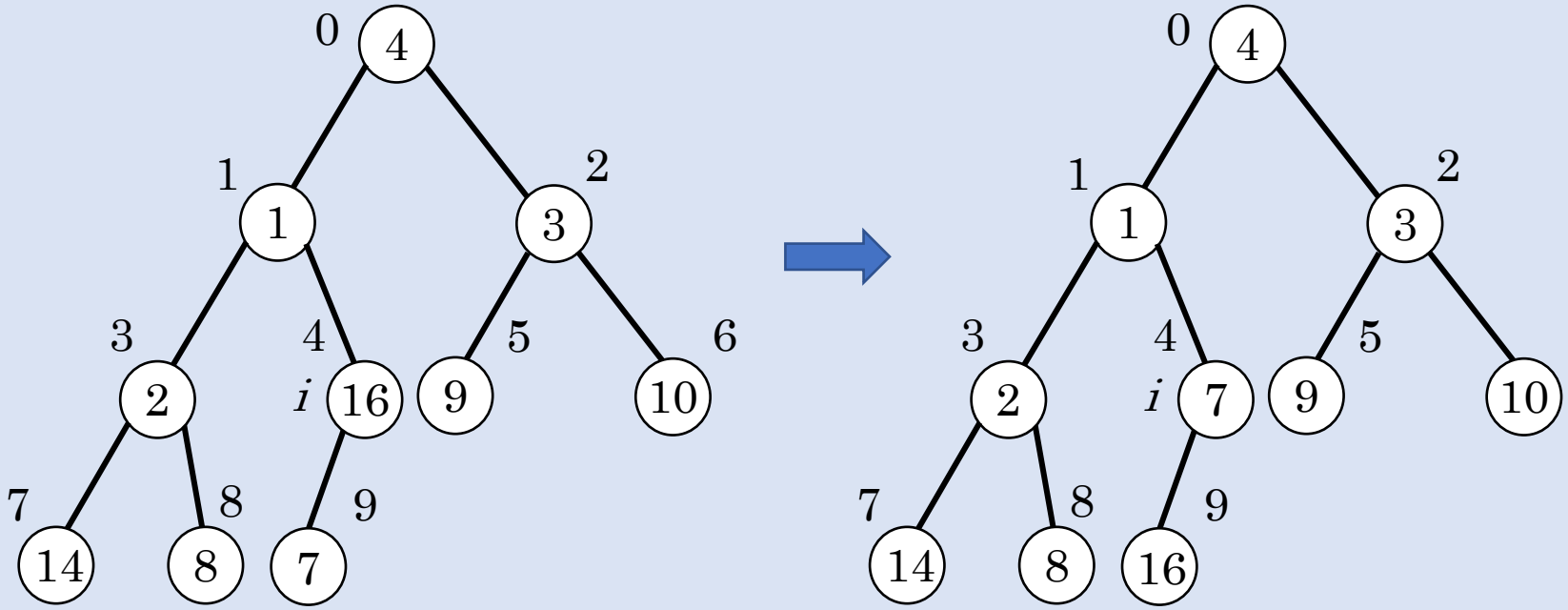
Note: h is the height of the binary tree and n is the number of nodes.

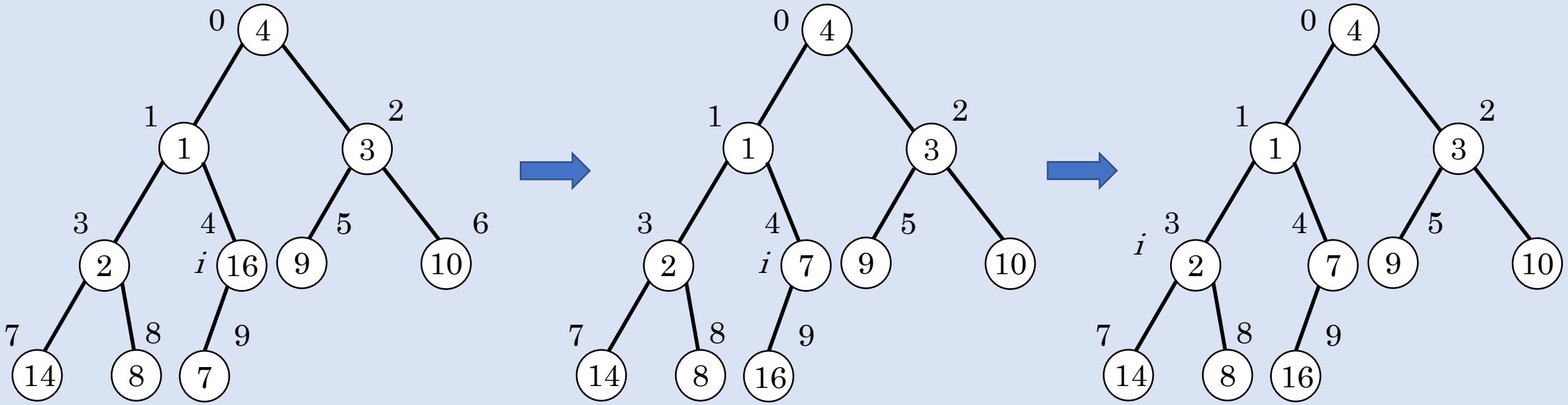
Building a Min-Heap

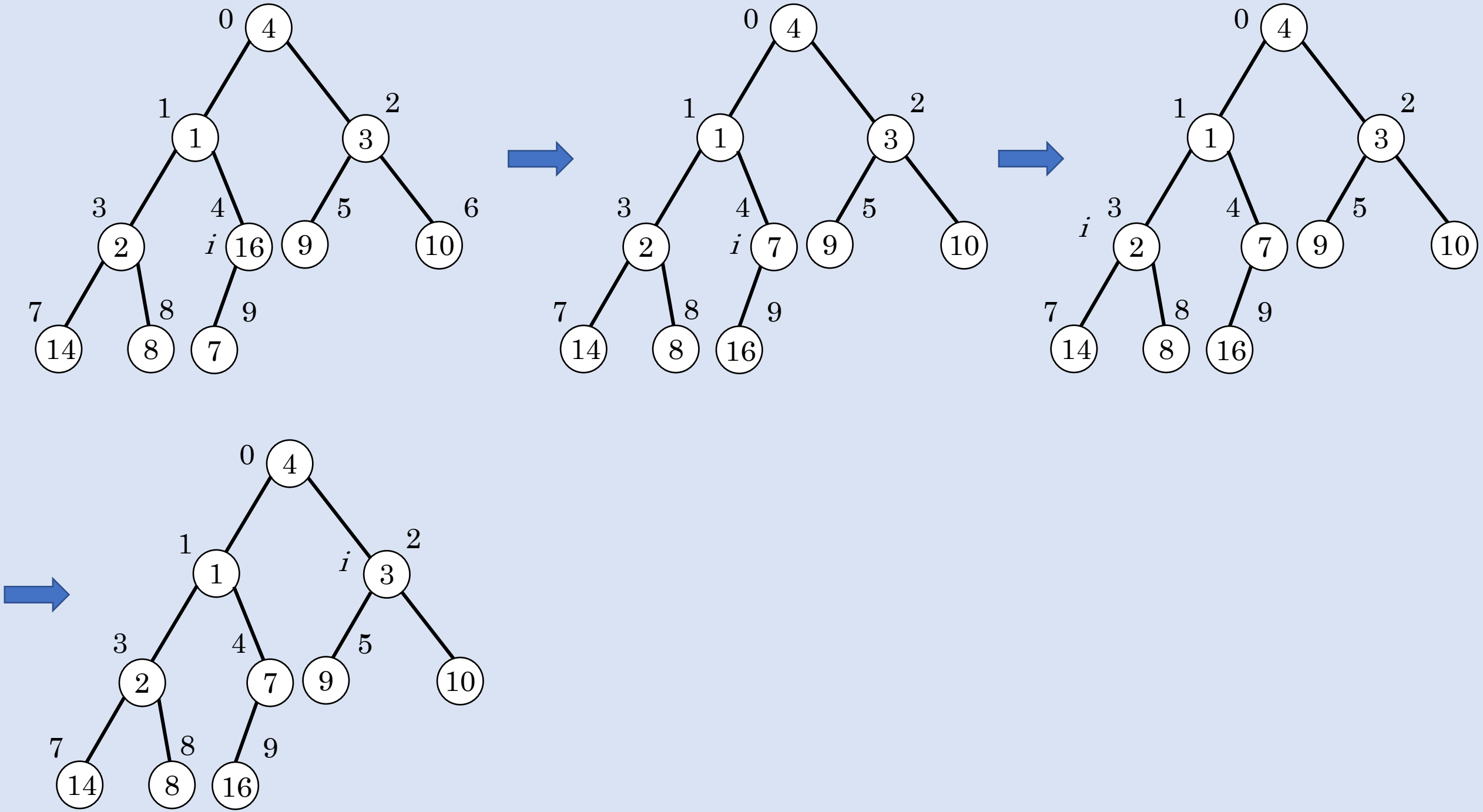
- Heapify works in a bottom-up manner to convert the array into a min-heap.
- The idea is that we go through any remaining nodes of the binary tree and invoke heapify for each one.

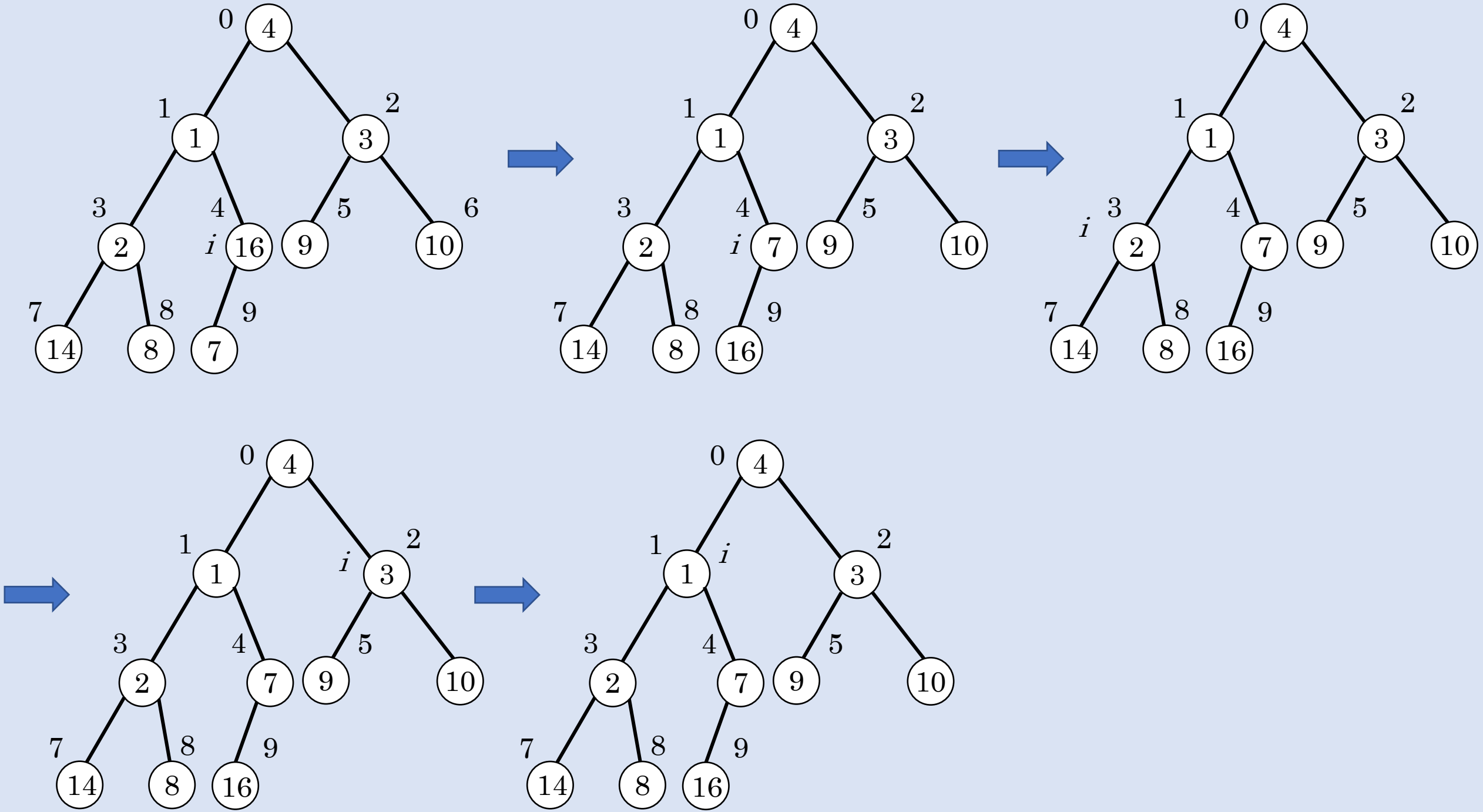
```
void buildMinHeap(int * arr, int n)
{
    for(int i = n / 2 - 1; i >= 0; --i)
        heapify(arr, n, i);
}
```

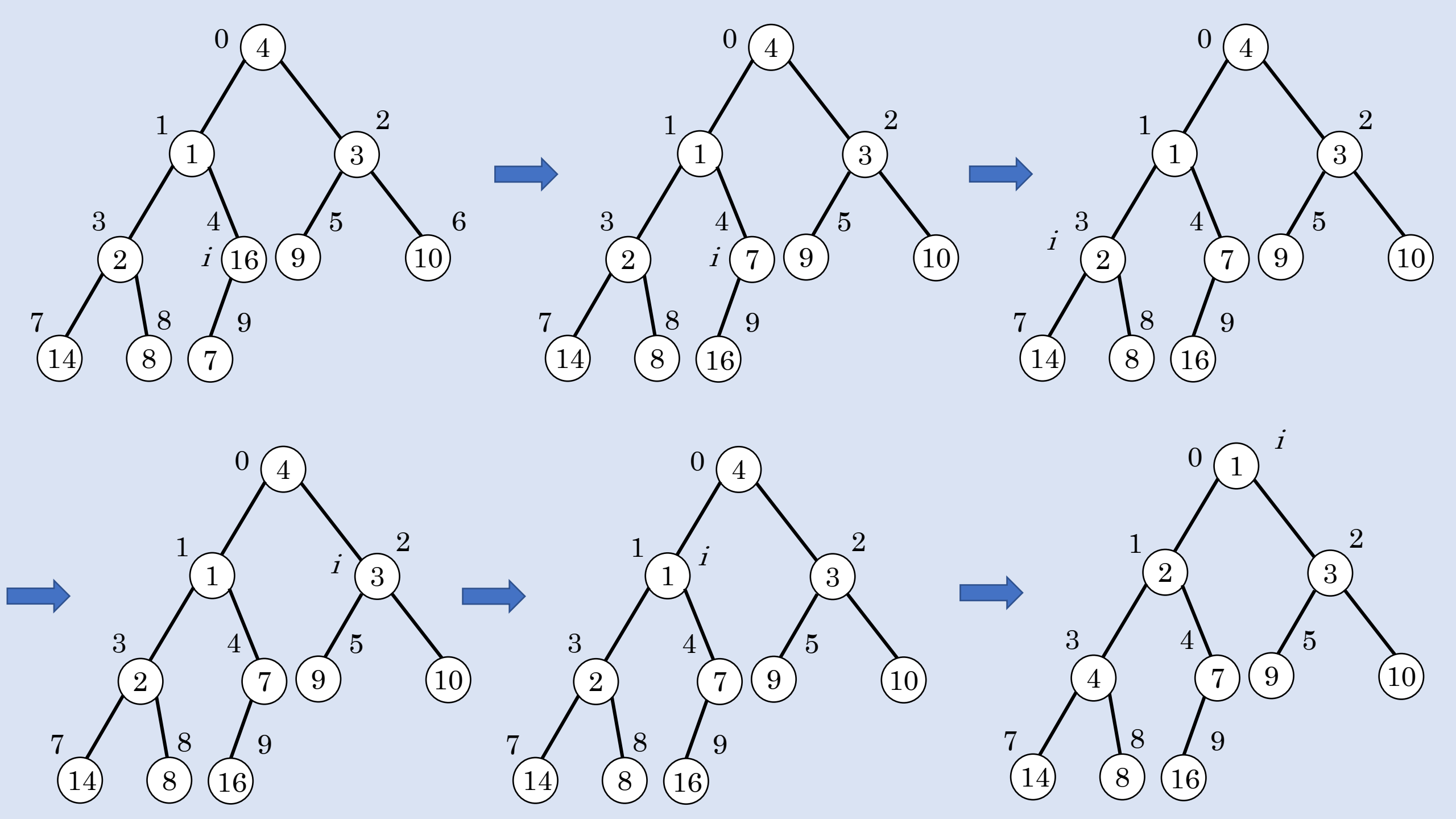

0
4
1
1
2
3
3
2
4
i
16
5
9
6
10
7
14
8
8
9
7


0
4
1
1
2
3
3
2
4
i
16
5
9
6
10
7
14
8
8
9
7
0
4
1
1
2
3
3
2
4
i
7
5
9
10
7
14
8
8
16
9


0
4
1
1
2
3
3
2
4
i
16
5
9
6
10
7
14
8
8
9
7
0
4
1
1
2
3
3
2
4
i
7
5
9
10
7
14
8
8
9
16
0
4
1
1
2
3
i
3
2
4
7
5
9
10
7
14
8
8
9
16


0 4
1 1
2 3
3 2
4 i 16
5 9
6 10
7 14
8 8
9 7
0 4
1 1
2 3
3 2
4 i 7
5 9
10
7 14
8 8
9 16
0 4
1 1
2 3
i 3 2
4 7
5 9
10
7 14
8 8
9 16
0 4
1 1
i 2 3
3 2
4 7
5 9
10
7 14
8 8
9 16


0 4
1 1
2 3
3 2
4 i 16
5 9
6 10
7 14
8 8
9 7
0 4
1 1
2 3
3 2
4 i 7
5 9
10
7 14
8 8
9 16
0 4
1 1
2 3
i 3 2
4 7
5 9
10
7 14
8 8
9 16
0 4
1 1
i 2 3
3 2
4 7
5 9
10
7 14
8 8
9 16
0 4
1 1
i 2 3
3 2
4 7
5 9
10
7 14
8 8
9 16


0 4
1 1
2 3
3 2
4 i 16
5 9
6 10
7 14
8 8
9 7
0 4
1 1
2 3
3 2
4 i 7
5 9
10
7 14
8 8
9 16
0 4
1 1
2 3
i 3 2
4 7
5 9
10
7 14
8 8
9 16
0 4
1 1
i 2 3
3 2
4 7
5 9
10
7 14
8 8
9 16
0 4
1 1
i 2 3
3 2
4 7
5 9
10
7 14
8 8
9 16
0 1
i
1 2
2 3
3 4
4 7
5 9
10
7 14
8 8
9 16

The Correctness of buildMinHeap

- **Initialization**: Before the first iteration of node $i = \lfloor (n-1)/2 \rfloor$. $\lfloor (n-1)/2 \rfloor + 1$, $\lfloor (n-1)/2 \rfloor + 2$,… *n* are some leaf of a potential max-heap.
- **Loop Invariant**: at the start of each iteration *i* of the for loop, each node $i+1, i+2, \ldots, n-1$ is the root of a max-heap. This means the subtrees rooted at the respective node. Inside the loop, heapify is applied at the sub roots which results in preserving the heap property.
- **Termination**: When $i < 0$, each node $0 - n - 1$ is the root of a min-heap. Most importantly, node 0 is the main root.

Running Time Analysis of buildMinHeap

```
void buildMinHeap(int * arr, int n)
{
    for(int i = n / 2 - 1; i >= 0; --i)
        heapify(arr, n, i);
}
```

$c_1 n/2 \rightarrow c_1 n$

$c_2 nlogn$

$$T(n) = c_1 n + c_2 nlogn \rightarrow O(nlogn)$$

Heap Applications

- There are 2 applications we will observe utilizing the heap data structure.
- Heapsort
- Priority Queues

Heapsort

- The Sorting Problem
  - Input: a sequence of n numbers $< a_1, a_2, \ldots, a_n >$
  - Output: a permutation (reordering) of the numbers in ascending order $< a_1{}', a_2{}', \ldots, a_n{}' >$ where $a_1' \leq a_2' \leq, \ldots, \leq a_n{}'$
  - We have a detailed discussion on a variety of sorting algorithms later in the semester. ☺
- Applying some of the functions we have learned, we can design the heapsort algorithm.
  - **Super Random Fun Fact**: This is Dr. Steinberg's favorite sorting algorithm. When he learned this back long ago he always assumed "heapify" was some Harry Potter Spell which made the learning even more fun and perhaps magical!

```
void heapSort(int * arr, int n)
{
    buildMinHeap(arr, n);

    for(int i = n - 1; i >= 0; --i)
    {
      swap(arr, 0, i);
      heapify(arr, i, 0);
    }

    //reverse the array to have it in ascending order
    int temp;

    int start = 0;
    int end = n - 1;

    while(start < end)
    {
        temp = arr[start];
        arr[start] = arr[end];
        arr[end] = temp;
        ++start;
        --end;
    }
}
```

Heapsort Running Time

```
void heapSort(int * arr, int n)
{
    buildMinHeap(arr, n);

    for(int i = n - 1; i >= 0; --i)
    {
      swap(arr, 0, i);
      heapify(arr, i, 0);
    }

    //reverse the array to have it in ascending order
    int temp;

    int start = 0;
    int end = n - 1;

    while(start < end)
    {
        temp = arr[start];
        arr[start] = arr[end];
        arr[end] = temp;
        ++start;
        --end;
    }
}
```

$c_1 nlogn$

$c_2 n$

$c_3 n$

$c_4 n \log n$

c_5

$c_6 n$

$$T(n) = c_1 nlogn + c_2 n + c_3 n + c_4 n \log n + c_5 + c_6 n \rightarrow O(nlogn)$$

Why does the heapsort have a reverse component?

- Since we are utilizing the min-heap property, the heapsort would result in a sorted array in descending order (largest to smallest).
- The reverse component has a linear runtime impact. The overall analysis still comes out to linearithmic.
- If the max-heap property was utilized, then the heapsort would result in the array being sorted in ascending order.

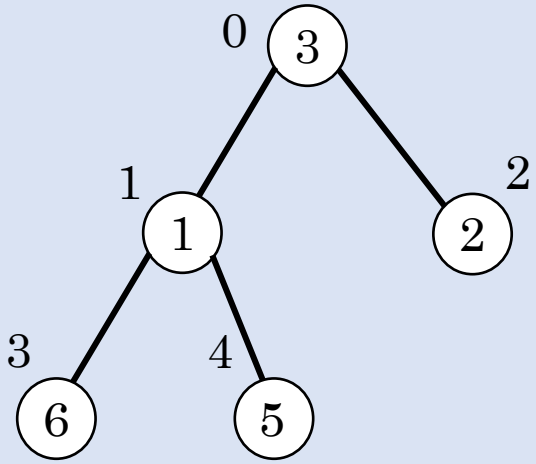
Heapsort Example

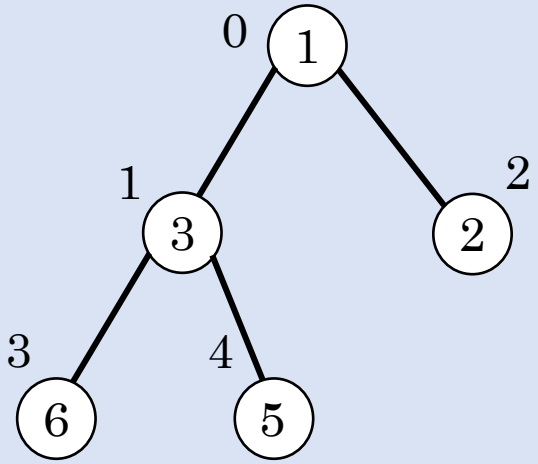
Apply the Heapsort algorithm to the following array.

3	1	2	6	5

Heapsort Example

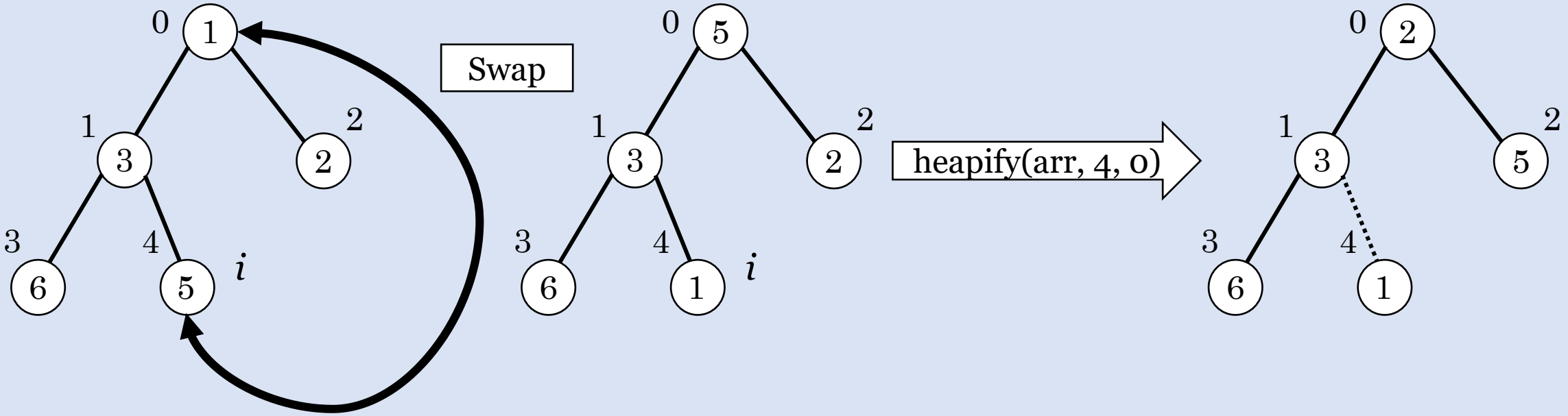
3	1	2	6	5





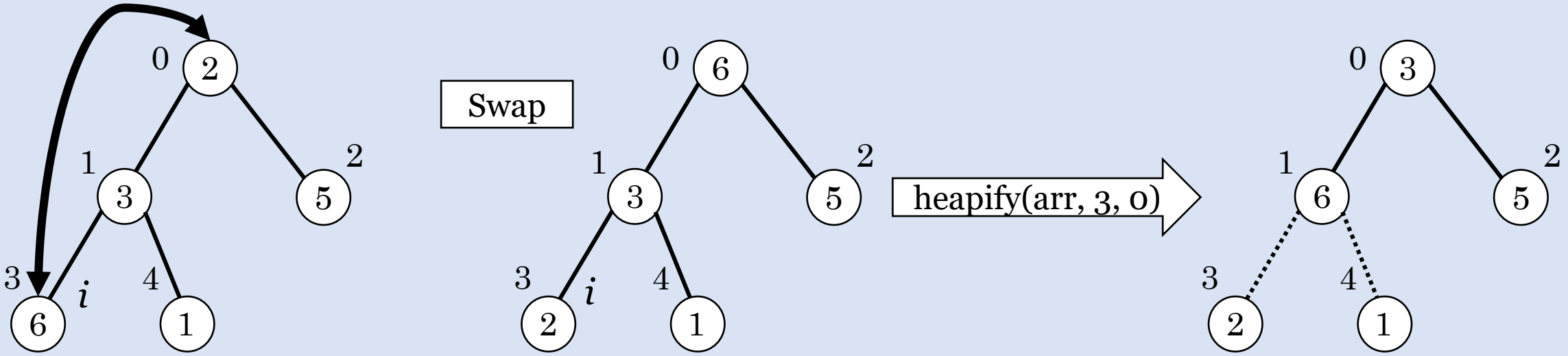
Heapsort Example

3	1	2	6	5



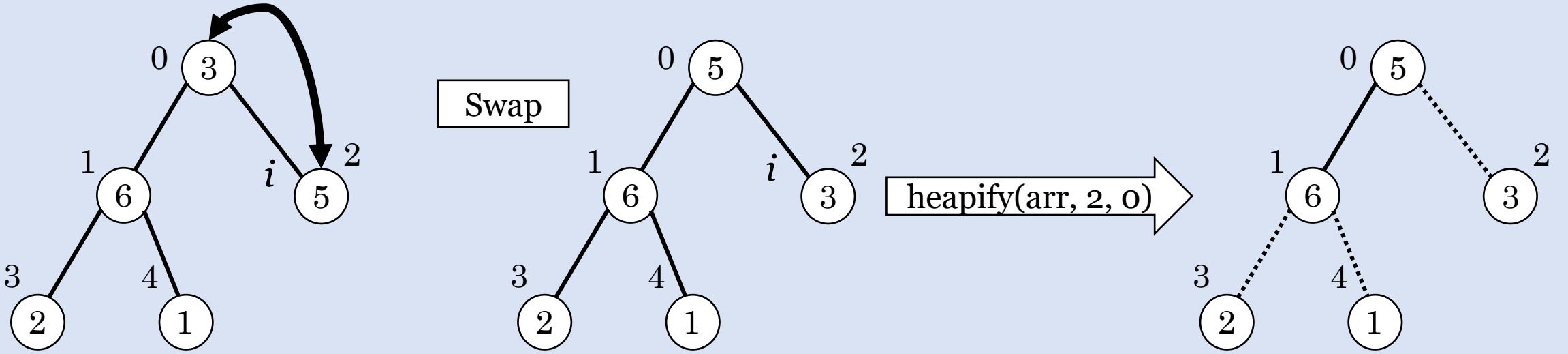
Heapsort Example

3	1	2	6	5



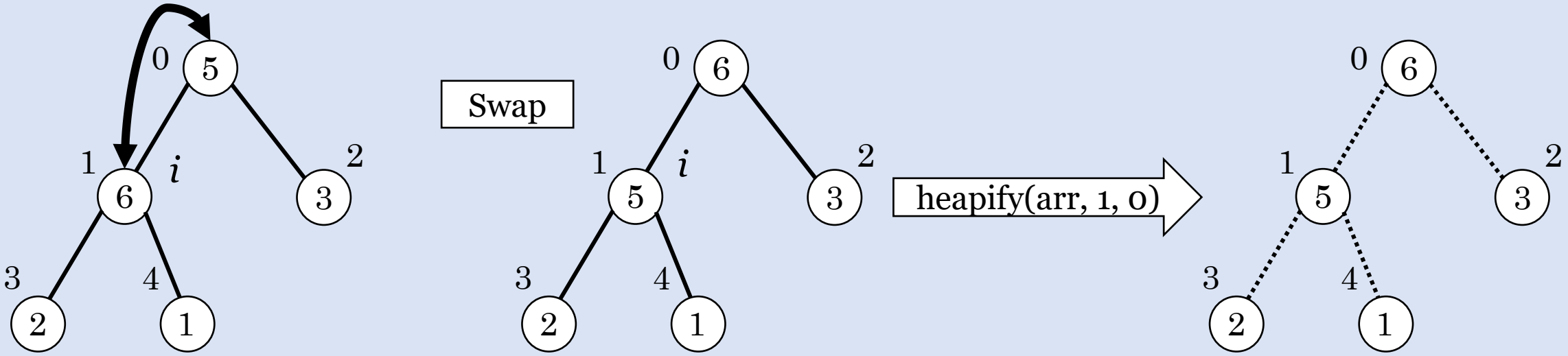
Heapsort Example

3	1	2	6	5



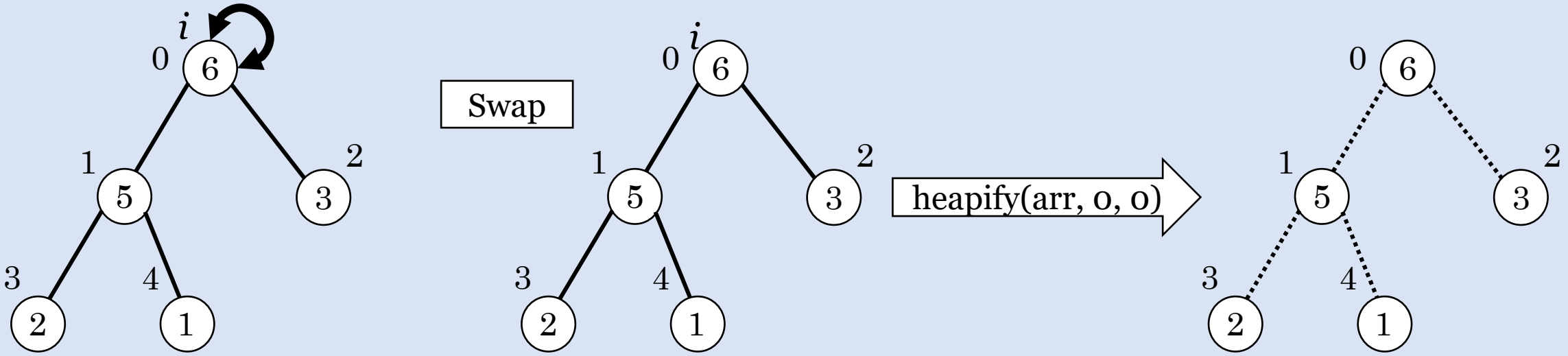
Heapsort Example

3	1	2	6	5



Heapsort Example

3	1	2	6	5



6	5	3	2	1

Reverse Array

1	2	3	5	6

Priority Queues

- Priority queue is a data structure used to maintain some set S of elements, where each element has a key.
- This extends our elementary queue data structure.
- Each key has a priority level component.
  - The priority levels are used to maintain the heap property.
- We simply take the concept of the heap property and give the values an extra special meaning (priority level).
  - This creates the **priority** queue!

Maximum-Priority Queue

- Maximum-Priority Queue Operations
  - Insert(Q,k): inserts the key k into the priority queue Q
  - Maximum(Q): returns the element in the queue Q with the largest key value
  - Extract-Maximum: like Maximum(Q) except it removes the element from the front of priority queue.
  - Increase-Key(Q,x,k): increase the value of element *x's* key to the new value k. Make sure the new value k exceeds the old key value.
- This uses the max-heap property!

Minimum-Priority Queue

- Minimum-Priority Queue Operations
  - Insert(Q,k): inserts the key k into the queue Q
  - Minimum(Q): returns the element in the queue Q with the smallest key value
  - Extract-Minimum: like Minimum(Q) except it removes the element from the front of priority queue.
  - Decrease-Key(Q,x,k): decrease the value of element *x's* key to the new value k. Make sure the new value k is smaller than the old key value.
- This uses the min-heap property!

Runtime Analysis of Priority Queues

- Using the binary trees, priority queues use the same operations as the heap data structure that were presented in these slides.
- This means that all operations besides getMinimum runs in logarithmic time!
  - getMinimum is still constant time.